

Name : Rudra Pratap Singh

SAP ID : 590032811

Batch : 16

Date : 11 September 2026

1. Aim

To design and implement a menu-driven C program using the switch-case construct that performs digit based operations on a positive integer entered by the user. The program must check whether the number is a Strong number, an Armstrong number, a Harshad number, or a Duck number. Each menu option is implemented as a separate user-defined function of void return type, called with a simple function call (no parameters). Each function independently reads the required input, processes it, and displays the result.

2. Algorithm

2.1 Main Menu Algorithm

1. Start the program.
2. Display a menu with five options: Strong Number, Armstrong Number, Harshad Number, Duck Number, and Exit.
3. Read the user's choice into an integer variable choice.
4. Using a switch statement on choice:
 - case 1: call checkStrongNumber()
 - case 2: call checkArmstrongNumber()
 - case 3: call checkHarshadNumber()
 - case 4: call checkDuckNumber()
 - case 5: print an exit message
 - default: print an 'invalid choice' message
5. Repeat steps 2-4 in a do-while loop until choice equals 5.
6. Stop.

2.2 checkStrongNumber()

A Strong number is a number whose sum of the factorials of its digits equals the number itself (e.g. $145 = 1! + 4! + 5!$).

1. Read a positive integer num; store it in original.
2. Initialise sum = 0.
3. While num > 0, repeat:
 - Extract the last digit: $\text{digit} = \text{num} \% 10$
 - Compute factorial of digit using a for loop
 - Add the factorial to sum
 - Remove the last digit: $\text{num} = \text{num} / 10$
4. If sum equals original, print that it is a Strong number, else print that it is not.

2.3 checkArmstrongNumber()

An Armstrong number is a number equal to the sum of its own digits, each raised to the power of the total number of digits (e.g. $153 = 1^3 + 5^3 + 3^3$).

1. Read a positive integer num; store it in original.
2. Count the total number of digits, count, using a loop.
3. While the number still has digits left, repeat:
 - Extract the last digit

- Raise it to the power count
- Add the result to sum
- Remove the last digit

4. If sum equals original, print that it is an Armstrong number, else print that it is not.

2.4 checkHarshadNumber()

A Harshad (Niven) number is a number that is exactly divisible by the sum of its digits (e.g. 18 $\rightarrow 1+8 = 9$, and $18 \% 9 = 0$).

1. Read a positive integer num; store it in original.
2. Initialise digitSum = 0.
3. While num > 0, add the last digit to digitSum and remove it from num.
4. If original is exactly divisible by digitSum, print that it is a Harshad number, else print that it is not.

2.5 checkDuckNumber()

A Duck number is a number that contains the digit 0 in it, but not as its leading digit. Since the user always enters a plain positive integer (whose first digit is never 0), the check reduces to: does 0 appear anywhere among the digits after the first one?

1. Read a positive integer num; store it in original.
2. Count the total number of digits, count.
3. Compute divisor = $10^{(\text{count}-1)}$ so that num / divisor gives the leading digit.
4. Take the remaining digits as num % divisor, and scan each of them.
5. If any remaining digit equals 0, set a flag hasZero = 1.
6. If hasZero is 1, print that it is a Duck number, else print that it is not.

3. Test Case Table

The program was compiled with gcc (with -Wall -Wextra, producing no warnings) and executed against the following 16 test cases, covering normal cases, boundary cases (single-digit numbers), and an invalid menu choice. All test cases produced the expected output.

S.No	Menu Option	Input	Test Objective	Expected / Actual Output	Result
1	Strong Number	145	Strong number	145 is a STRONG number.	Pass
2	Strong Number	123	Not a strong number	123 is NOT a Strong number.	Pass
3	Strong Number	1	Strong number (boundary: single digit)	1 is a STRONG number.	Pass
4	Strong Number	40585	Strong number (5-digit)	40585 is a STRONG number.	Pass
5	Armstrong Number	153	Armstrong number	153 is an ARMSTRONG number.	Pass
6	Armstrong Number	123	Not an Armstrong number	123 is NOT an Armstrong number.	Pass
7	Armstrong Number	9474	Armstrong number (4-digit)	9474 is an ARMSTRONG number.	Pass
8	Armstrong Number	9475	Not an Armstrong number	9475 is NOT an Armstrong number.	Pass
9	Harshad Number	18	Harshad number	18 is a HARSHAD number.	Pass
10	Harshad Number	19	Not a Harshad number	19 is NOT a Harshad number.	Pass
11	Harshad Number	1	Harshad number (boundary: single digit)	1 is a HARSHAD number.	Pass
12	Duck Number	105	Duck number (0 in middle)	105 is a DUCK number.	Pass
13	Duck Number	10	Duck number (0 at end)	10 is a DUCK number.	Pass
14	Duck Number	123	Not a Duck number	123 is NOT a Duck number.	Pass
15	Duck Number	7	Not a Duck number (single digit)	7 is NOT a Duck number.	Pass
16	Invalid Menu Choice	6	Invalid choice handled by default case	Invalid choice! Please enter a number between 1 and 5.	Pass

4. Source Code

File: digit_operations.c

```

/* =====
Program   : Menu-driven program for Digit Based Operations
Author    : Rudra Pratap Singh
SAP ID    : 590032811
Batch     : 16
Date      : 11 September 2026

Description:
Performs the following digit based checks on a positive
integer entered by the user, using a switch-case menu.
    1. Strong Number
    2. Armstrong Number
    3. Harshad Number
    4. Duck Number
Each option is handled by a separate void function that
takes no parameters. Every function reads its own input
from the keyboard, processes it, and prints the result.
===== */

#include <stdio.h>

/* Function prototypes - void return type, no parameters */
void checkStrongNumber(void);
void checkArmstrongNumber(void);
void checkHarshadNumber(void);
void checkDuckNumber(void);

int main(void)
{
    int choice;

    do
    {
        printf("\n==== DIGIT BASED OPERATIONS MENU =====\n");
        printf("1. Check Strong Number\n");
        printf("2. Check Armstrong Number\n");
        printf("3. Check Harshad Number\n");
        printf("4. Check Duck Number\n");
        printf("5. Exit\n");
        printf("Enter your choice (1-5): ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                checkStrongNumber();
                break;
            case 2:
                checkArmstrongNumber();
                break;
            case 3:
                checkHarshadNumber();
                break;
            case 4:
                checkDuckNumber();
                break;
            case 5:
                printf("Exiting program. Goodbye!\n");
                break;
            default:
                printf("Invalid choice! Please enter a number between 1 and 5.\n");
        }

    } while (choice != 5);

    return 0;
}

```

```

/* -----
Function : checkStrongNumber
Purpose  : A number is a Strong Number if the sum of the
           factorial of its digits is equal to the number
           itself. Example: 145 = 1! + 4! + 5! = 1+24+120
           ----- */
void checkStrongNumber(void)
{
    int num, original, digit, i, fact;
    long sum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &num);
    original = num;

    while (num > 0)
    {
        digit = num % 10;

        /* calculate factorial of the digit */
        fact = 1;
        for (i = 1; i <= digit; i++)
            fact = fact * i;

        sum = sum + fact;
        num = num / 10;
    }

    if (sum == original)
        printf("%d is a STRONG number.\n", original);
    else
        printf("%d is NOT a Strong number.\n", original);
}

/* -----
Function : checkArmstrongNumber
Purpose  : A number is an Armstrong Number if the sum of
           its own digits, each raised to the power of the
           total number of digits, equals the number itself.
           Example: 153 = 1^3 + 5^3 + 3^3
           ----- */
void checkArmstrongNumber(void)
{
    int num, original, digit, temp, count = 0, i;
    long sum = 0;
    long power;

    printf("Enter a positive integer: ");
    scanf("%d", &num);
    original = num;

    /* count number of digits */
    temp = num;
    while (temp > 0)
    {
        count++;
        temp = temp / 10;
    }

    temp = num;
    while (temp > 0)
    {
        digit = temp % 10;

        /* raise digit to the power 'count' */
        power = 1;
        for (i = 1; i <= count; i++)
            power = power * digit;

        sum = sum + power;
    }
}

```

```

        temp = temp / 10;
    }

    if (sum == original)
        printf("%d is an ARMSTRONG number.\n", original);
    else
        printf("%d is NOT an Armstrong number.\n", original);
}

/* -----
Function : checkHarshadNumber
Purpose  : A number is a Harshad (Niven) Number if it is
           exactly divisible by the sum of its digits.
           Example: 18 -> 1+8=9, 18 % 9 == 0
----- */
void checkHarshadNumber(void)
{
    int num, original, digit, temp;
    int digitSum = 0;

    printf("Enter a positive integer: ");
    scanf("%d", &num);
    original = num;
    temp = num;

    while (temp > 0)
    {
        digit = temp % 10;
        digitSum = digitSum + digit;
        temp = temp / 10;
    }

    if (digitSum != 0 && original % digitSum == 0)
        printf("%d is a HARSHAD number.\n", original);
    else
        printf("%d is NOT a Harshad number.\n", original);
}

/* -----
Function : checkDuckNumber
Purpose  : A number is a Duck Number if it contains the
           digit 0 in it, but the leading (first) digit is
           not zero. Since the user enters a plain positive
           integer, the leading digit can never be zero, so
           we simply check whether any digit equal to 0
           appears among the remaining digits.
           Example: 3040, 7005 are Duck numbers; 12345, 102
           -> 102 is a Duck number (contains a 0, not leading)
----- */
void checkDuckNumber(void)
{
    int num, original, digit, temp, count, divisor, k;
    int hasZero = 0; /* flag: zero found among non-leading digits */

    printf("Enter a positive integer: ");
    scanf("%d", &num);
    original = num;

    if (num == 0)
    {
        printf("0 is not considered, since a Duck number must be positive.\n");
        return;
    }

    /* count the total number of digits in num */
    count = 0;
    temp = num;
    while (temp > 0)
    {
        count++;
        temp = temp / 10;
    }

```

```
/* find 'divisor' = 10^(count-1) so that num/divisor gives the
   leading digit, and num % divisor gives the remaining digits */
divisor = 1;
for (k = 1; k < count; k++)
    divisor = divisor * 10;

temp = num % divisor; /* all digits except the leading one */

/* check each of the remaining digits for a 0 */
for (k = 0; k < count - 1; k++)
{
    digit = temp % 10;
    if (digit == 0)
        hasZero = 1;
    temp = temp / 10;
}

if (hasZero)
    printf("%d is a DUCK number.\n", original);
else
    printf("%d is NOT a Duck number.\n", original);
}
```

5. Compilation and Sample Execution

The program was compiled using GCC on a Linux terminal and executed interactively. A representative run (one pass through each menu option, plus an invalid choice) is shown below; the complete set of 16 test runs is summarised in the test case table in Section 3.

```
$ gcc -Wall -Wextra -o digit_operations digit_operations.c
$ ./digit_operations
```

```
===== DIGIT BASED OPERATIONS MENU =====
```

1. Check Strong Number
2. Check Armstrong Number
3. Check Harshad Number
4. Check Duck Number
5. Exit

```
Enter your choice (1-5): 1
Enter a positive integer: 145
145 is a STRONG number.
```

```
===== DIGIT BASED OPERATIONS MENU =====
```

```
Enter your choice (1-5): 2
Enter a positive integer: 153
153 is an ARMSTRONG number.
```

```
===== DIGIT BASED OPERATIONS MENU =====
```

```
Enter your choice (1-5): 3
Enter a positive integer: 18
18 is a HARSHAD number.
```

```
===== DIGIT BASED OPERATIONS MENU =====
```

```
Enter your choice (1-5): 4
Enter a positive integer: 105
105 is a DUCK number.
```

```
===== DIGIT BASED OPERATIONS MENU =====
```

```
Enter your choice (1-5): 6
Invalid choice! Please enter a number between 1 and 5.
```

```
===== DIGIT BASED OPERATIONS MENU =====
```

```
Enter your choice (1-5): 5
Exiting program. Goodbye!
```

6. Conclusion

A menu-driven C program was successfully designed and implemented using the switch-case construct to check whether a positive integer is a Strong number, an Armstrong number, a Harshad number, or a Duck number. Each check was written as an independent void function, invoked with a plain function call, that performs its own input, processing, and output. The program was compiled without any errors or warnings and was verified against 16 test cases, including boundary and invalid-input cases, all of which produced the expected results.